

Task 4 Jenkins installation and setup, explore the environment.

Jenkins is the "leading open-source automation server."

- It's built in Java.
- Install **jdk 11 or jdk 17 or jdk 21**
- Set the path for jdk Click on Search ,Type "Edit System Environment Variable" then click on Environment variable >Select path> click on Edit >Paste the path copied from program file- java folder- jdk 11 or 17 or 21 -select the jdk and click on bin -copy the path till bin -Paste the path on new folder >Click ok>click ok>click ok>.
- Install Jenkins from <https://www.jenkins.io/download/>

Jenkins Version 2.474

Select the Windows option it will download .

- It's built in Java.
- It's basically a platform for the Software Development Life Cycle (SDLC).
- It's commonly used to implement continuous integration and continuous delivery concepts specifically.
- It's useful to anyone who works on a software project, including:
 1. Application Developers
 2. Web Developers
 3. IT Operations
 4. Systems Engineers
 5. DevOps Engineers

Jenkins is known for being easy to use and highly adaptable. There are thousands of customizations and plugins that make Jenkins suitable for most use cases. It also works in the most common OS environments and is relatively lightweight (doesn't use a lot of resources).

General Terms

Below are some commonly used abbreviations and terms that you'll notice:

- CD: CD stands for Continuous Delivery unless specified otherwise. It could also be Continuous Deployment.
- CI: Continuous Integration
- DevOps: It's really an organizational culture. The concept of developers and operations working in harmony. CI/CD is a common strategy used, but DevOps encompasses much more.
- DSL: Domain Specific Language
- GUI: Graphical User Interface
- Job: A task configured in Jenkins; It's synonymous with Project. "Job" is now the deprecated term.
- JSON: lightweight data-interchange format that uses Key/Value pairs
- md5sum: The digital fingerprint of a file (128-bit MD5 hashes)
- Pipeline: The job type created by the Pipeline plugin; It's used more generically in CI/CD discussion.
- Repo: Repository
- SCM: source code (or control) management
- SDLC: Software development life cycle
- WebGUI: Graphical user interface in a web browser
- XML: A metalanguage that allows users to define custom markup languages

Key Concepts of CI/CD/Jenkins

Continuous Integration/Delivery Concepts

Continuous Integration

A software development practice where contributors are integrating their work very frequently. This results in multiple daily integrations to a mainline. Automated testing (post-commit promotion) is commonly used.

Basic Workflow

- Checkout from an SCM (like git)
- Branch and make local changes
- Add or change tests as necessary
- Trigger automated build locally
- If successful, consider committing
- Update with latest from mainline
- Push changes, build and test on integration machine

Best Practices

- Maintain a single source repository.
- Have a common mainline Branch (usually master).
- Automate the Build.
- Build every commit.
- Prioritize fixing broken builds.
- Keep your builds fast!
- Minimize potential for user error, automate everything possible
- Make the Build Self-testing.
- Self-testing code
- Use Unit Tests for granular functionality.
- Everyone commits frequently (at least daily, preferably).
- Communication is key! Seems basic, but it's where many orgs fail.
- Frequent merges will help avoid conflict errors.
- The working branch should be updated as frequently as possible to help avoid very large diffs while merging.
- Testing environment should be as close to production as possible
- Make it easy for anyone to get the latest.
- Keep it open; everyone should see what's happening.
- Automate the deployment.

Continuous Delivery

A software development discipline where software is built so that it CAN be released to production at any time.

Difference between CI and CD

- Continuous Delivery is the ability to release at any time.
- Continuous Integration is just the practice of integrating code continuously. It doesn't necessarily mean that it can or will be released at any time.

Stages of CI and CD

Build -> Deploy -> Test -> Release

Installing/Configuring Jenkins

Installation Wizard

- Utility in WebGUI that allows for some basic configuration and the installation of common plugins
- After installing the package and starting the service via the CLI, the installation wizard becomes available from the Jenkins WebGUI.
- <https://localhost:8080/> or <http://localhost:8888>
- Secret password in `/var/lib/jenkins/secrets/initialAdminPassword`
- Can add an admin user and set credentials

Then, go back to the WebGUI and start again. Set admin settings, user, password, Press "Enter." Click "Start Using Jenkins."

Configuring Jenkins

Plugins

Plugins add additional functionality to Jenkins.

- Can be used to add functionality in many ways
- "Credentials" manages user Credentials.
- "Git" and "GitHub" manage Git integrations.
- "JUnit" handles unit testing reports.
- There are thousands of plugins.
- Plugins (.hpi files) of various releases can be downloaded from:
- <http://updates.jenkins-ci.org/download/plugins>

Plugin Manager

The UI to install and configure plugins in the Jenkins WebGUI

Procedure for Installing Plugins

1. From the Dashboard, click "Manage Jenkins."

2. Click "Manage Plugins."
3. Click the "Available" tab.
4. Filter plugins for the targeted plugin with the search box.
5. Click the "Install" button for the desired plugin.

Procedure for Removing Plugins

1. From the Dashboard, click "Manage Jenkins."
2. Click "Manage Plugins."
3. Click the "Installed" tab.
4. Click the "Uninstall" button for the desired plugin.

Procedure for Downgrading Plugins

1. From the Dashboard, click "Manage Jenkins."
2. Click "Manage Plugins."
3. Click the "Installed" tab.
4. Click the "Downgrade to..." button for the desired plugin.

Procedure for Upgrading Plugins

1. From the Dashboard, click "Manage Jenkins."
2. Click "Manage Plugins."
3. Click the "Updates" tab.
4. Check the box next to the desired update.
5. Click "Download now and install after restart."
6. Restart the service jenkins service from the Jenkins master cli, or check the box

that will do that for you.

(Advanced) Installing a Plugin from a .hpi File

1. From the Dashboard, click "Manage Jenkins."
2. Click "Manage Plugins."
3. Click the "Advanced" tab.

4. In the "Upload Plugin" section, use the "Choose File" button to navigate to your hpi file.

5. Install it

The Dashboard of Jenkins

Left Panel

→ New Item

- This is where you'll add projects, folders and pipelines.
- The core of Jenkins functionality is here.

→ People

- This is where you can see a list of users and their latest activities.

→ Build History

- You'll see an overview display of build history for all projects in graphical form.

→ Manage Jenkins

- Where Jenkins is managed

→ My Views

- For configuring custom views for projects for the logged in user.

→ Credentials

- Lists credentials that have been configured for Jenkins

→ Build Queue

Jobs waiting for an executor are listed here.

→ Build Executor Status

- The status of projects associated with Jenkins executors
- An executor runs projects dictated by Jenkins.
- They can run in parallel.
- The default number of executors on the master is 2.

→ **Build History**

- Shows the history of builds
- Stable is blue
- Red is broken

→ **Authentication Versus Authorization**

Authentication identifies a user, while Authorization dictates what a user is allowed to do.

→ **Security Terms**

- Auditing: Tracking who did what on your Jenkins server
- Credentials: Username/Password for logging into Jenkins.

→ **Matrix Security**

- Allows you to grant specific permissions to users and groups.
- Most Common Permissions: Overall, Slave, Job, Run, View, and SCM.

→ **Configuration Procedure**

1. Go to the Jenkins Dashboard.

<http://<your-server>:8080>

2. Click "Manage Jenkins."
3. Click "Configure Global Security."
4. Check "Enable Security."
5. Select "Jenkins' own user database."
6. Place a check mark next to "Allow users to sign up."
7. Select "Matrix-based Security."
8. Select the admin user and check all the permissions.
9. Click Save.
10. (Optional) Set "Read" permissions for "Anonymous."
11. (Optional) Set options appropriate for other users.

→ Types of Projects

Freestyle Project

- This is the most versatile type of project.
- You can use any SCM with any build system.
- It can even be leveraged for non-build tasks.

Pipeline

- Formerly known as workflow
- Long running activity orchestration
- Can span multiple slaves
- Jenkins pipeline is a suite of plugins which supports CI/CD pipelines into Jenkins.
- "Pipeline as code" is illustrated by the Jenkins file using the Jenkins DSL(Domain specific language).

External Job

Allows for management of job execution in another automation system

→ Multi-Configuration Project

- For projects that have a lot of different configurations
- It can be best if you need to test in multiple different types of environments.
- This is likely more suitable for projects that are comprised of many build steps.

→ Folder

- Creates a container with nested items
- It creates a separate namespace for the nested projects.
- Projects can be organized in folders.

→ GitHub Organization

Scans a GitHub organization for repositories and branches

→ Multibranch Pipeline

Detects branches in an SCM repository

→ Upstream Versus Downstream Projects

- A project that is triggered by another project is considered to be downstream of that project.
- A project that triggers another project is considered to be upstream of that project.

→ Builds

A build can describe what is produced from a project, or a single run of a project.

→ Build Terms

- Artifact: Something produced from a build
- Build Step: A single action in a build
- Repository: The home of the sources code
- Slave: An agent or node that runs builds
- Trigger: A mechanism that starts a build

→ Build Steps

- Build steps are added as part of the project configuration.
- They can be specified and configured from a Jenkins file in pipeline configurations.

→ Examples of Build Steps:

- Execute shell can be a groovy script
- Execute a command in a Docker container
- Execute an Ant/Maven Build
- Running Tests
- Deploying

→ Jenkins Build Environment Variables

- Environment variables available to all builds
- Accessible via this syntax:

String Interpolation

"\${VARIABLE}"

#OR

\$VARIABLE

- BUILD_NUMBER: Just the Jenkins build number; might be useful to interpolate into an artifact name.
- NODE_NAME: The node name as defined in the "Manage Nodes" section in "Manage Jenkins."

This might be useful when sending alerts, and could help with problem node identification.

- JOB_NAME: Name of the job
- EXECUTOR_NUMBER: Number of the executor on the node
- WORKSPACE: The fully-qualified path to the workspace; can be useful in some

Scripts

Build Triggers

Pushing versus Pulling

- When Jenkins reaches out to the source to determine what has happened, that's a "pull" scenario.
- When the source notifies Jenkins (like with a Githook), that's a "push" scenario.
- Pulling is inherently passive. There is the potential for latency at the rate of the polling interval.
- Pushing is generally faster and preferred.

SCM Polling

- Jenkins will poll your SCM (like Git or Subversion) on an interval.
- Described as a "pull" mechanism as opposed to "push."
- If there is a change for the specified ref(s) or branch(es), the project will run.

- If using the Jenkins Git plugin with GitHub, you can select this field and leave it empty; the Githook will trigger builds on changes. This is actually using the SCM Polling setting for a "push" scenario.

- The interval setting uses the same cron format as described in the "Time Interval" trigger below.

→ Build Periodically

- Builds on a pre-defined interval
- Schedule Config Item
- Same cron format is used for "SCM Polling"
- "H" stands for hash.
- The H symbol can be used with a range. For example,

H H(0-7) * * * ...means some time between 12:00 AM (midnight) to 7:59 AM.

- The "H" symbol can be thought of as a random value over a range, but it actually is a hash of the job name, not a random function, so that the value remains stable for any given project.

- Uses the "cron" date format

* * * * *

| | | | |

| | | | +---Day of the Week (range: 1-7, 1 standing for Monday)

| | | +-----Month of the Year (range: 1-12)

| | +-----Day of the Month (range: 1-31)

| +-----Hour (range: 0-23)

+-----Minute (range: 0-59)

Examples:

every fifteen minutes (perhaps at :07, :22, :37, :52)

H/15 * * * *

every ten minutes in the first half of every hour (three times, perhaps at :04, :14, :24)

H(0-29)/10 * * * *

once every two hours at 45 minutes past the hour starting at 9:45 AM and finishing at 3:45 PM every weekday.

45 9-16/2 * * 1-5

once in every two hours slot between 9 AM and 5 PM every weekday (perhaps at 10:38 AM, 12:38 PM, 2:38 PM, 4:38 PM)

H H(9-16)/2 * * 1-5

once a day on the 1st and 15th of every month except December

H H 1,15 1-11 *

Runs every year

@yearly

OR

@annually

Runs every month

@monthly

Runs every week

@weekly

Runs every day

@daily

Runs everyday some time between 12:00am to 12:59am

@midnight

Runs every hour

@hourly

→ Triggered by Another Project

- This can be set to watch for the completion of another project. (A "pull" scenario)

- Triggers can also be "pushed" by adding a build step "Trigger/call builds on other projects."
- With the "Parameterized Trigger Plugin", you can trigger, and pass parameters to, another project as a "Post-build" step.

→ Hooks

- Hooks can be configured to trigger jobs via Jenkins' REST API.
- Hooks provide a "push" scenario. The source informs Jenkins when to build.
- Git hooks can be configured in the more traditional way as well, by adding scripts to the ".git/hooks" directory in the repo path.
- Both the GitHub and Git plugins provide a hook inherently.
- Prerequisites for the procedures below:
 - Have SSH keys configured for Jenkins and GitHub
 - Your Jenkins master must be publicly routable, reachable from the central GitHub servers.
 - The Project must be previously configured to monitor the GitHub project already in the "Source Code Management" section.

→ Procedure for Configuring the GitHub Plugin Git Hook

1. Log in to GitHub.
2. Within GitHub, Click your project.
3. Click "Settings" from the project view.
4. Click "Integrations and Services."
5. Find "Jenkins (GitHub plugin)", via search.
6. Set the following as the "Jenkins hook url":
 http://<your-server-hostname>.<your-domain>:<port>/github-webhook/
 # e.g. http://brandon4231.mylabserver.com:8080/github-webhook/
7. On your Jenkins Master Dashboard, log in to Jenkins with Administrator User.
8. Click your project.

9. Click 'Configure' from the left panel.
10. Under build triggers, check "GitHub hook trigger for GITScm polling."
11. Click "Save."
12. (Optional) Make a commit to the repo and wait for the build to trigger to test.
13. (Optional) On GitHub: From the "Jenkins (GitHub plugin)" view in the "Integrations and Services" section, there is an option to test. Click "Test Service" from there and it will attempt to trigger a build from there without committing to the repo.

→ Procedure for Configuring the Git Plugin Git Hook

1. Log in to GitHub.
2. On GitHub, click your project.
3. Click "Settings" from the project view.
4. Click "Integrations and Services."
5. Find "Jenkins (Git plugin)", via search.
6. Add your Jenkins master url, including port (probably 8080):
`http://<your-server-hostname>.<your-domain>:<port>/`
e.g. `http://brandon4231.mylabserver.com:8080/`
7. On your Jenkins Master Dashboard, log in to Jenkins with Administrator User.
8. Click your project.
9. Click 'Configure' from the left panel.
10. Under build triggers, check "Poll SCM."
11. Leave the "Schedule" empty.
12. Click "Save."
13. (Optional) Make a commit to the repo and wait for the build to trigger to test

→ Artifacts and Fingerprints

Artifacts

Items produced from a build.

- Can download from WebGUI
- Add a post-build step to generate a artifact for a project.

→ Adding an Artifact to a Freestyle Project

1. In the "Project Configuration" view, go to the "Post-build Actions" section.
2. Click the "Add post-build action" dropdown and select "Archive the artifacts."
3. In the Files to archive field, insert a pattern or path to the file(s) you wish to archive.

→ Copying Artifacts

Can leverage the "Copy Artifact" Plugin to move artifacts between projects/jobs

→ Procedure for Copying Artifacts

1. Install the "Copy Artifact" plugin.
2. In the "Project Configuration" view, go to the "Build" section.
3. Click the "Add build step" dropdown, and select "Copy artifacts from another project."
4. Insert the "Project name" of the project you'd like to copy the artifact from.
5. Select the appropriate option for "Which build."
 - "Latest successful build" is the default.
6. Select the "Artifacts to copy" or "Artifacts not to copy."
7. Specify the "Target directory" from within the workspace for the artifacts to land.

→ Setting Artifact Retention Policy

1. In the "Project Configuration" view, go to the "General" section.
2. Check "Discard old builds."

3. Click "Advanced."
4. Specify "Days to keep artifacts."
5. Specify "Max # of builds to keep with artifacts."

→ Fingerprints

Tracks md5sum of artifacts that are used between projects.

→ Viewing Fingerprints of Artifacts

1. In the "Project Build" view, click "See Fingerprints."
2. From there you can see the Artifacts and their original owning projects.
3. If you click "More Details," you can see everywhere where that specific file has been used.
4. You can also see the md5 hash for the file.

→ Distributed Builds

- Running builds on one or more slaves instead of a master-only configuration.
- The master is the controller, while the slaves run projects.
- Distributing builds is a way to lower the resource load on the master.
- As the scope of a Jenkins configuration expands with more and more projects, it may make sense to incrementally add build slaves.

→ About Build Slaves

- Synonymous with Build Agent
- Triggered by ssh agent by default
- SSH, JNLP and Cloud are the different agent types.
- Once set up, the distribution of tasks between nodes is fairly automatic.
- Adding slaves doesn't really change how you interact with Jenkins much at all.
- The master will still serve all administrative tasks, http requests.

→ Executors

Executors dictate how many builds can run on a node at a given time.

- Executors can still exist on the master after you configure build slaves.

- "# of executors" can be set by selecting the configuration icon (gear) in the "Manage Nodes" list view for the node, and then setting the number in the corresponding field.

→ Assigning Nodes to Projects

Specific build slaves can be selected for projects with expression matching in the "Project Configuration" view, or in the Jenkins file for Pipeline applications.